

```

%{                                // SECTION 1 Declarations for C-Bison
#include <stdio.h>
#include <ctype.h>                // tolower()
#include <string.h>               // strcmp()
#include <stdlib.h>               // exit()

#define FF fflush(stdout);        // to force immediate printing

int yylex () ;
void yyerror (char *) ;
char *my_malloc (int) ;

// Not needed using Direct Translation:
char *gen_code (char *) ;
char *int_to_string (int) ;
char *char_to_string (char) ;
char *token ;
char temp [2048] ;

// Definitions for explicit attributes

typedef struct s_attr {
    int value ;    // - Numeric value of a NUMBER
    char *code ;   // - to pass IDENTIFIER names, and other translations
} t_attr ;

#define YYSTYPE t_attr

```

```
#define YYDEBUG 1
```

```
%}
```

```
%token NUMBER
```

```
%token IDENTIF
```

```
%token STRING
```

```
%token SETQ
```

```
%token SETF
```

```
%token DEFUN
```

```
%token IF
```

```
%token PROGN
```

```
%token LOOP
```

```
%token WHILE
```

```
%token DO
```

```
%token LE
```

```
%token GE
```

```
%token NEQ
```

```
%token AND
```

```
%token OR
```

```
%token NOT
```

```
%token PRINT
```

```
%token PRINC
```

```
%token MOD
```

```
%token MAIN
```

```
%%
```

```

program : /* empty */
        | top_forms
        ;

top_forms : top_form
        | top_form top_forms
        ;

top_form : var_decl
        | func_decl
        | '(' MAIN ')'
        ;

var_decl: '(' SETQ IDENTIF
        expr ')'
        ;

func_decl: '(' DEFUN MAIN '(' params ')' { char delims[] = " ";
        token = strtok($5.code, delims);
        while (token != NULL){
            printf("variable %s\n", token);
            token = strtok(NULL, delims);
        }
        printf(": %s ", $3.code); }
        block ')'
        | '(' DEFUN IDENTIF '(' params ')' { char delims[] = " ";
        token = strtok($5.code, delims);

```

```

                                while (token != NULL){
                                    printf("variable %s\n", token);
                                    token = strtok(NULL, delims);
                                }
                                printf(": %s ", $3.code);
                                printf("( %s-- )\n", $5.code);

                                token = strtok($5.code, delims);
                                while (token != NULL){
                                    printf("%s !\n", token);
                                    token = strtok(NULL, delims);
                                } }

                                block ')'
                                ;

params : /* empty */                                { $$code = gen_code(""); }
        | param_list                                { $$code = $1.code; }
        ;

param_list : IDENTIF                                { ; }
            | IDENTIF param_list                    { ; }
            ;

func_params: expr                                    { ; }
            | expr func_params                       { ; }
            ;

block : statement                                    { ; }

```

```

    | statement block                { ; }
;

function_call: '(' IDENTIF func_params ')' { printf("%s\n", $2.code); }
    | '(' IDENTIF ')'                   { printf("%s\n", $2.code); }

statement : '(' SETF IDENTIF expr ')' { printf("%s !\n", $3.code); }
    | '(' PRINT STRING ')'           { printf(".\" %s\" cr\n", $3.code); }
    | '(' PRINC STRING ')'           { printf(".\" %s\" cr\n", $3.code); }
    | '(' PRINC expr ')'             { printf(". \n"); }
    | '(' PROGN printf_sequence ')' { printf("cr\n"); }
    | if_start if_body               { printf("else\n"); }
    | if_body ')'                   { printf("then\n"); }
    | if_start if_body ')'           { printf("then\n"); }
    | '(' LOOP WHILE                 { printf("begin "); }
    | expr                          { printf(" while \n"); }
    | DO block ')'                  { printf("repeat\n"); }
    | function_call                  { ; }
;

printf_sequence : princ_stmt          { ; }
    | princ_stmt printf_sequence { ; }
;

princ_stmt : '(' PRINC expr ')'       { printf(". "); }
    | '(' PRINC STRING ')'           { printf(".\" %s\" ", $3.code); }
;

```

[illegible]

```

        | IDENTIF                                { printf("%s @ ", $1.code); }
        ;

%%

int current_line = 1;

void yyerror(char *msg)
{
    fprintf(stderr, "%s at line %d\n", msg, current_line);
    printf("\n");
}

char *int_to_string(int n)
{
    sprintf(temp, "%d", n);
    return gen_code(temp);
}

char *char_to_string(char c)
{
    sprintf(temp, "%c", c);
    return gen_code(temp);
}

char *my_malloc(int size)
{
    char *ptr;

```

```

static long int total_bytes = 0;
static int alloc_calls = 0;

ptr = malloc(size);
if (ptr == NULL) {
    fprintf(stderr, "No memory for %d more bytes\n", size);
    fprintf(stderr, "Allocated %ld bytes in %d calls\n", total_bytes, alloc_calls);
    exit(0);
}
total_bytes += (long) size;
alloc_calls++;

return ptr;
}

typedef struct s_reserved_word {
    char *name;
    int token;
} t_reserved_word;

t_reserved_word reserved_table[] = {
    "main",      MAIN,
    "setq",      SETQ,
    "setf",      SETF,
    "defun",     DEFUN,
    "if",        IF,
    "progn",     PROGN,
    "loop",      LOOP,

```

```
"while",      WHILE,
"do",         DO,
"<=",         LE,
">=",         GE,
"/=",         NEQ,
"and",        AND,
"or",         OR,
"not",        NOT,
"print",      PRINT,
"princ",      PRINC,
"mod",        MOD,
NULL,         0
```

```
};
```

```
t_reserved_word *lookup_reserved(char *sym_name)
```

```
{
```

```
    int i = 0;
```

```
    t_reserved_word *ptr;
```

```
    ptr = reserved_table;
```

```
    while (ptr[i].name != NULL) {
```

```
        if (strcmp(ptr[i].name, sym_name) == 0) {
```

```
            return &(ptr[i]);
```

```
        }
```

```
        i++;
```

```
    }
```

```
    return NULL;
```

```
}
```

```
char *gen_code(char *src)
{
    char *dest;
    int len;

    len = strlen(src) + 1;
    dest = (char *) my_malloc(len);
    strcpy(dest, src);

    return dest;
}

int yylex()
{
    int i;
    unsigned char c;
    unsigned char next;
    char ops_list[] = "!<=|>%&/+-*";
    char tmp_str[256];
    t_reserved_word *kw_entry;

    do {
        c = getchar();

        if (c == '#') {
            do {
                c = getchar();
```

```
        } while (c != '\n');
    }

    if (c == '/') {
        next = getchar();
        if (next != '/') {
            ungetc(next, stdin);
        } else {
            c = getchar();
            if (c == '@') {
                do {
                    c = getchar();
                    putchar(c);
                } while (c != '\n');
            } else {
                while (c != '\n') {
                    c = getchar();
                }
            }
        }
    }
    } else if (c == '\\') {
        c = getchar();
    }

    if (c == '\n')
        current_line++;

} while (c == ' ' || c == '\n' || c == 10 || c == 13 || c == '\t');
```

```
if (c == '\\') {
    i = 0;
    do {
        c = getchar();
        tmp_str[i++] = c;
    } while (c != '\\' && i < 255);

    if (i == 256) {
        printf("WARNING: string longer than 255 chars at line %d\n", current_line);
    }
    tmp_str[--i] = '\\0';
    yylval.code = gen_code(tmp_str);
    return STRING;
}

if (c == '.' || (c >= '0' && c <= '9')) {
    ungetc(c, stdin);
    scanf("%d", &yylval.value);
    return NUMBER;
}

if ((c >= 'A' && c <= 'Z') || (c >= 'a' && c <= 'z')) {
    i = 0;
    while ((c >= 'A' && c <= 'Z') || (c >= 'a' && c <= 'z') ||
        (c >= '0' && c <= '9') || c == '_') && i < 255) {
        tmp_str[i++] = tolower(c);
        c = getchar();
    }
}
```

```
    }  
    tmp_str[i] = '\\0';  
    ungetc(c, stdin);  
  
    yylval.code = gen_code(tmp_str);  
    kw_entry = lookup_reserved(yylval.code);  
  
    if (kw_entry == NULL) {  
        return IDENTIF;  
    } else {  
        return kw_entry->token;  
    }  
}  
  
if (strchr(ops_list, c) != NULL) {  
    next = getchar();  
    sprintf(tmp_str, "%c%c", (char) c, (char) next);  
    kw_entry = lookup_reserved(tmp_str);  
  
    if (kw_entry == NULL) {  
        ungetc(next, stdin);  
        yylval.code = NULL;  
        return c;  
    } else {  
        yylval.code = gen_code(tmp_str);  
        return kw_entry->token;  
    }  
}
```

```
    if (c == EOF || c == 255 || c == 26) {  
        return 0;  
    }  
  
    return c;  
}  
  
int main()  
{  
    yyparse();  
    return 0;  
}
```